Random Access File through Java

Random Access File

- Random access in a file enables us to access individual records directly without searching through other records.
- The heart of the random access file mechanism is the class called *java.io.RandomAccessFile*.
- It has all the capabilities of *FileInputStream* and *FileOutputStream*.
- The instance of this class not only helps in creating or opening an existing file but also provide a parameter to specify the mode of operation such as read(r) or write(w) when opening a file.

Random Access File

- Visualize a random access file as an array of bytes stored in the file system.

- Records in the random access file can be directly accessed with the index number supplied through the *seek* method and subsequently read/write chunks of data from/to the file.
- Example:
  - file.seek(10);
  - file.writeInt(3);

Opening & Writing to File

```
import java.io.RandomAccessFile;
public class Main
{
        public static void main(String[] args) throws Exception
        {
        RandomAccessFile fileStore = new RandomAccessFile("test.bin", "rw");
        fileStore.seek(100); # sets the file-pointer offset at which the next read or write access
        fileStore.writeInt(5);
        fileStore.writeInt(7); # write an integer to the file as 4 bytes
        fileStore.close();
        }
}
```

File Opening Mode

Different File Opening Modes

Mode	Meaning
"r"	The file is opened in a read-only mode.
"rw"	The file is opened in a read-write mode. The file is created if it does not exist. → Flushes the contents of the file and the modification date when the file is closed
"rws"	The file is opened in a read-write mode. Any modifications to the file's content and its metadata are written to the storage device immediately. → Flushes the contents of the file and the modification date of the file immediately
"rwd"	The file is opened in a read-write mode. Any modifications to the file's content are written to the storage device immediately. → Flushes the contents of the file, but the modification date might not change until the file is closed.

Opening & Read to File

```
import java.io.RandomAccessFile;
public class Main
{
        public static void main(String[] args) throws Exception
        {
                RandomAccessFile fileStore = new RandomAccessFile("test.bin", "rw");
                fileStore.seek(100);
                 System.out.println(fileStore.readInt()); // will show 5
                 fileStore.seek(0);
                 fileStore.seek(104); // seek 100 and 4 for the intger 5 so that we can read 7
                 System.out.println(fileStore.readInt()); // will show 7
                fileStore.close();
        }
}
```

A Sample Program

- Write a program to store the following two fixed-length records into a binary file:
  - First record: 1,2,3
  - Second Record: 7,8,9

Each Record has a size of **3** Integers

A Sample Program Cont.

```
import java.io.RandomAccessFile;
public class Main
{
        public static void main(String[] args) throws Exception
        {
        RandomAccessFile fileStore = new RandomAccessFile("test.bin", "rw");

        fileStore.writeInt(1);
        fileStore.writeInt(2);
        fileStore.writeInt(3);
        fileStore.writeInt(7);
        fileStore.writeInt(8);
        fileStore.writeInt(9);
        fileStore.close();

        }
}
```

A Sample Program Cont.

- Write a program to read the **second** record from the binary file previously written.

A Sample Program Cont.

```
import java.io.RandomAccessFile;
public class Main
{
	public static void main(String[] args) throws Exception
	{
		RandomAccessFile fileStore = new RandomAccessFile("test.bin", "rw");
		fileStore.seek(4*3);
		System.out.println(fileStore.readInt());
		System.out.println(fileStore.readInt());
		System.out.println(fileStore.readInt());
		fileStore.close();
	}
}
```

A Sample Program Cont.

- Write a program to replace “7” of the **second** record from the binary file previously written with “19”.

A Sample Program Cont.

```
import java.io.RandomAccessFile;
public class Main
{
public static void main(String[] args) throws Exception
{
	RandomAccessFile fileStore = new RandomAccessFile("test.bin", "rw");
	fileStore.seek(4*3);
	fileStore.writeInt(19);
	fileStore.seek(0);
	fileStore.seek(4*3);
	System.out.println(fileStore.readInt());
	System.out.println(fileStore.readInt());
	System.out.println(fileStore.readInt());
	fileStore.close();
}
}
```

A Sample Program Cont.

- Write a program to find out the number of integers previously written on binary file.

```
import java.io.RandomAccessFile;
public class Main
{
public static void main(String[] args) throws Exception
	{
		RandomAccessFile fileStore = new RandomAccessFile("test2.bin", "rw");
		System.out.println(fileStore.length()/4);
		fileStore.close();
	}
}
```